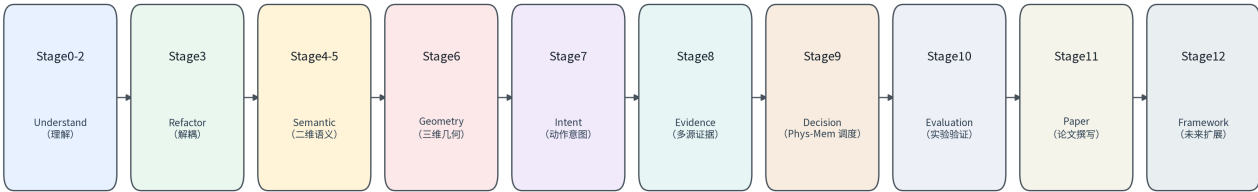


Stage0-Stage12 完整科研流程整理

以下内容按原文整理，正文不改写。

Stage0-Stage12 科研路线



这两部分实际上就是论文真正的核心贡献（Core Contribution）。

如果说：

Stage4 是 Semantic（语义）

Stage6 是 Physical（几何）

Stage7 是 Intent（动作）

那么：

Stage8 = 如何融合（Fusion）

Stage9 = 如何真正控制Memory（Phys-Mem）

注意，这里我和之前的设计相比做了一些修改。

Fusion不是论文贡献。

真正的贡献应该是：

Physics-aware Memory Scheduling

Fusion只是其中的一步。

所以Stage9才是整篇论文真正的Method。

Stage8 Multi-Evidence Fusion（多证据融合）

一、阶段目标（Goal）

经过前面的开发：

现在已经有四种Evidence：

ORB

↓

Appearance Evidence

LightGlue

↓

Semantic Evidence

Depth

↓

Geometry Evidence

Action

↓

Intent Evidence

但是：

现在：

四种Evidence彼此独立。

真正的问题变成：

Memory到底应该相信谁？

所以：

这一阶段：

不是：

写算法。

而是：

建立：

统一决策框架。

二、本阶段论文目标

对应：

Method

第四节：

Multi-Evidence Memory Fusion

论文重点：

提出：

新的：

Memory Evidence。

即：

Appearance

+

Semantic

+

Geometry

+

Intent

↓

Unified Memory Evidence

而不是：

四个算法。

三、本阶段开发目标

新增：

FusionEngine

注意：

Fusion：

不是：

Memory Scheduler。

它只是：

Evidence Aggregator。

即：

Similarity

↓

Evidence

↓

Fusion

↓

Memory Score

不要：

直接：

Decision。

四、允许修改范围

允许：

新增：

FusionEngine

FusionConfig

FusionWeight

FusionBenchmark

允许：

修改：

接口。

禁止：

修改：

Memory Scheduler。

五、禁止修改

禁止：

修改：

Memory Scheduler

Memory Buffer

KV Cache

禁止：

修改：

ORB

LightGlue

Depth

Action。

全部保持。

六、Codex Prompt（完整）

任务：

开始设计：

Multi-Evidence Fusion。

当前系统已经拥有：

Appearance

Semantic

Geometry

Intent

四种Evidence。

不要修改：

ORB。

LightGlue。

Depth。

Action。

=====

目标：

新增：

FusionEngine。

负责：

统一融合：

所有Evidence。

=====

输入：

appearance_similarity

semantic_similarity

geometry_similarity

intent_state

intent_confidence

=====

要求：

不要采用：

固定if else。

设计：

统一接口。

支持：

Weighted Fusion。

Rule-based。

Future Learned Fusion。

方便以后继续扩展。

=====

输出：

Unified Memory Score

Evidence Confidence

Evidence Weight

=====

要求：

FusionConfig。

所有权重：

可配置。

支持：

动态调整。

=====

新增：

Visualization。

包括：

Weight Distribution

Evidence Curve

Evidence Contribution

=====

Benchmark：

统计：

不同Evidence：

贡献率。

=====

实验：

设计：

① ORB

② LightGlue

③ Depth

④ Action

⑤ Fusion

分别比较：

Memory Stability。

Frame Consistency。

Runtime。

=====

最后：

总结：

为什么：

Fusion

不是简单加权？

为什么：

Future可以继续加入：

VGGT

DUST3R

Metric3D？

不要实现：

Memory Decision。

七、最终输出

Codex：

需要输出：

① FusionEngine

② FusionConfig

③ WeightManager

④ Fusion Visualization

⑤ Fusion Benchmark

⑥ Evidence分析

⑦ Framework分析

八、Checklist

必须得到：

☒ Evidence统一接口

☒ Weight配置

☒ Fusion可视化

☒ Contribution分析

必须回答：

为什么：

Fusion

不能直接：

Replace Memory?

回答不了。

Stage8结束不了。

九、论文对应

对应：

Method

Multi-Evidence Fusion

这是论文：

第二张Method图。

十、Git建议

commit:

Stage8

Fusion Engine Added

十一、下一阶段

进入：

Phys-Mem。

Stage9 Phys-Mem (Physics-aware Memory Scheduling)

这一阶段：

才是真正的论文。

如果Reviewer问：

你的创新是什么？

真正回答：

不是：

Depth。

不是：

Action。

不是：

Fusion。

而是：

我们重新定义了Memory Scheduling。

这才是论文。

一、阶段目标

重新设计：

StableWorld：

Memory Scheduler。

目前：

StableWorld：

Similarity

↓

Threshold

↓

Keep / Delete

太简单。

现在：

需要：

Evidence

↓

Fusion

↓

Physics-aware Memory Scheduler

↓

Memory State

二、本阶段论文目标

对应：

Method：

第五节：

Physics-aware Memory Scheduling

这就是：

论文标题。

真正对应的方法。

三、本阶段开发目标

新增：

PhysMemScheduler

替换：

MemoryScheduler

注意：

不是：

删除。

而是：

继承。

即：

MemoryScheduler

↓

PhysMemScheduler

保持：

所有接口一致。

四、允许修改范围

允许：

修改：

Memory Scheduler。

允许：

新增：

PhysMemScheduler

MemoryPolicy

DecisionRule

StateMachine

五、禁止修改

禁止：

修改：

ORB

LightGlue

Depth

Action

Fusion

全部：

保持。

六、Codex Prompt（完整）

任务：

开始实现：

Physics-aware Memory Scheduling。

当前：

已经存在：

FusionEngine。

现在：

不要修改：

Fusion。

不要修改：

ORB。

不要修改：

Depth。

不要修改：

Action。

=====

目标：

实现：

PhysMemScheduler。

继承：

MemoryScheduler。

=====

输入：

Unified Memory Score

Geometry Confidence

Intent State

Memory Buffer

=====

输出：

Memory State。

包括：

KEEP

REFRESH

INSERT

REPLACE

EVICT

=====

要求：

设计：

State Machine。

不要：

if else堆积。

=====

要求：

Memory Policy。

支持：

Rule。

Score。

Future RL。

=====

新增：

Memory Timeline。

新增：

Decision Timeline。

新增：

State Transition Graph。

=====

Benchmark：

统计：

Keep

Replace

Refresh

Evict

比例。

=====

实验：

设计：

① StableWorld

② Semantic

③ Physical

④ Fusion

⑤ Phys-Mem

比较：

Memory Stability。

Frame Consistency。

Memory Replacement。

KV Update。

=====

最后：

请总结：

为什么：

Phys-Mem

相比：

StableWorld

真正改变的是：

Memory Scheduling?

而不是：

Similarity。

七、最终输出

Codex：

需要输出：

① PhysMemScheduler

- ② MemoryPolicy
- ③ Decision Engine
- ④ State Machine
- ⑤ Timeline
- ⑥ Benchmark
- ⑦ StableWorld

VS

Phys-Mem

完整实验

八、Checklist

必须得到：

☒ KEEP

☒ REPLACE

☒ REFRESH

☒ INSERT

☒ EVICT

五种状态。

必须回答：

为什么：

Phys-Mem

属于：

新的Memory Framework?

而不是：

Depth增强版?

如果回答不了。

Stage9不能结束。

九、论文对应

对应：

Method

Physics-aware Memory Scheduling

Core Algorithm

这一节就是整篇论文最重要的一节，也是论文标题所对应的内容。

十、Git建议

commit:

Stage9

Phys-Mem Scheduler Implemented

十一、进入下一阶段

进入 Stage10 (Evaluation)，这一阶段不再新增算法，而是围绕 StableWorld → LightGlue → Depth → Action → Fusion → Phys-Mem 构建统一的实验平台，包括消融实验、对比实验、可视化、统计分析和论文图表生成。这里的目标是把前面所有工作转化为可发表论文的实验结果，让每一个创新点都有对应的数据支撑。

给出stage10-11

下面这两个阶段其实已经不是开发算法了，而是完成论文。

如果前面 Stage0~9 是：

Research + Engineering

那么：

Stage10~11 就是：

Science + Paper

这也是很多论文最后失败的地方：

算法做好了，但是实验不会做，论文不会讲。

所以这两个阶段我进行了重新设计，不仅是跑实验，而是按照 CCF-B/B+ Reviewer 的角度去设计。

Stage10 Evaluation Framework（统一实验平台）

一、阶段目标（Goal）

从这一阶段开始：

停止增加新算法。

以后：

任何代码修改：

只能：

修Bug。

不能：

继续创新。

因为：

论文需要：

固定版本。

这一阶段：

唯一目标：

建立：

完整实验平台。

二、本阶段论文目标

完成：

Experiment

全部内容。

包括：

Implementation Details

↓

Comparison

↓

Ablation

↓

Visualization

↓

Discussion

以后：

论文实验。

全部来自这里。

三、本阶段开发目标

建立：

统一：

Experiment Framework。

支持：

Baseline

↓

ORB

↓

LightGlue

↓

LightGlue+Penalty

↓

Depth

↓

Depth+Action

↓

Fusion

↓

Phys-Mem

统一运行。

统一统计。

统一画图。

统一导出。

四、允许修改范围

允许：

新增：

evaluation/

benchmark/

visualization/

logger/

analysis/

允许：

新增：

CSV。

图片。

日志。

实验脚本。

五、禁止修改

禁止：

修改：

ORB

LightGlue

Depth

Action

Fusion

PhysMem

算法：

全部冻结。

六、Codex Prompt（完整）

任务：

开始建立：

统一实验平台。

当前：

算法已经全部完成。

不要继续修改任何算法。

=====

目标：

建立：

Evaluation Framework。

支持：

Baseline

ORB

LightGlue

Penalty

Depth

Action

Fusion

PhysMem

统一运行。

=====

新增：

ExperimentConfig。

支持：

一键切换：

Method。

=====

新增：

BenchmarkRunner。

自动：

运行所有实验。

=====

新增：

Logger。

保存：

Runtime。

Memory。

Similarity。

Depth。

Intent。

Decision。

=====
新增：

Visualization。

自动生成：

Figure。

Table。

Timeline。

=====
要求：

支持：

CSV

JSON

PNG

PDF

导出。

=====
实验：

请自动设计：

- ① Main Comparison
- ② Ablation Study
- ③ Sensitivity Analysis
- ④ Runtime Analysis
- ⑤ Memory Analysis
- ⑥ Failure Cases

=====
自动生成：

论文需要：

Figure。

包括：

Figure1

Framework

Figure2

Memory Timeline

Figure3

Similarity

Figure4

Depth

Figure5

Action

Figure6

Fusion

Figure7

Ablation

Figure8

Failure Cases

=====

自动生成：

Table。

包括：

Main Results。

Runtime。

Memory。

Ablation。

=====

最后：

输出：

完整实验目录结构。

不要修改算法。

七、最终输出

Codex:

输出:

- ① Evaluation Framework
- ② BenchmarkRunner
- ③ Logger
- ④ Visualization
- ⑤ CSV
- ⑥ Figure
- ⑦ Table
- ⑧ Experiment Report

八、Checklist

必须生成:

- ☒ Main Results
- ☒ Ablation
- ☒ Runtime
- ☒ Failure Cases
- ☒ Sensitivity
- ☒ Figures
- ☒ Tables

否则:

Stage10结束不了。

九、论文对应

对应:

Experiment

全部章节

包括:

4.1

Implementation

4.2

Comparison

4.3

Ablation

4.4

Runtime

4.5

Discussion

十、Git建议

建议：

commit:

Stage10

Evaluation Framework Completed

十一、下一阶段

进入：

Paper。

Stage11 Paper Construction（论文整理与投稿准备）

这是：

整个项目最后一阶段。

这一阶段：

停止开发。

全部工作：

围绕：

论文。

一、阶段目标

目标：

不是：

继续改代码。

而是：

完成：

CCF论文。

二、本阶段论文目标

完成：

完整：

论文。

包括：

Title

↓

Abstract

↓

Introduction

↓

Related Work

↓

Method

↓

Experiment

↓

Conclusion

全部完成。

三、本阶段开发目标

不开发。

只整理：

代码

↓

实验

↓

图片

↓

论文

四、允许修改范围

允许：

修改：

论文。

图片。

实验。

Latex。

Markdown。

五、禁止修改

禁止：

修改：

任何：

算法。

任何：

实验结果。

保证：

可重复。

六、Codex Prompt（完整）

任务：

开始整理：

论文。

当前：

代码已经冻结。

实验已经完成。

不要继续修改任何算法。

=====

目标：

根据：

整个项目。

自动整理：

论文。

=====

请生成：

Title。

要求：

符合：

CCF-B/B+

Computer Vision

Video Generation

Memory Scheduling

=====

生成：

Abstract。

包括：

Background。

Problem。

Method。

Experiment。

Contribution。

=====

生成：

Introduction。

要求：

按照：

Appearance

↓

Semantic

↓

Physical

↓

Intent

↓

PhysMem

Story。

=====

生成：

Related Work。

包括：

Long Video Generation。

Memory Mechanism。

Image Matching。

Depth Estimation。

World Model。

=====

生成：

Method。

包括：

System Overview。

Architecture。

Fusion。

PhysMem。

=====

自动引用：

Method Figure。

=====

生成：

Experiment。

自动引用：

所有：

Figure。

Table。

=====

生成：

Conclusion。

Future Work。

=====

最后：

总结：

论文：

三个核心贡献。

创新点。

不足。

未来工作。

=====

请输出：

符合CCF会议论文结构的Markdown和LaTeX版本。

七、最终输出

Codex：

输出：

① Paper.md

② Paper.tex

③ Figures

④ Tables

⑤ References

⑥ Appendix

⑦ Supplementary

八、Checklist

必须完成：

☒ 标题

☒ 摘要

☒ Introduction

☒ Related Work

☒ Method

☒ Experiment

☒ Conclusion

☒ Appendix

并确保：

论文所有图片、表格、实验数据均可追溯到 Stage10 自动生成的实验平台。

这样论文具有良好的可重复性。

九、论文对应

这一阶段对应整篇论文：

CCF-B/B+ Conference Paper

100%

建议最终组织结构如下：

1 Introduction

2 Related Work

3 Method

3.1 Framework

3.2 Semantic Memory

3.3 Physical-aware Memory

3.4 Intent-aware Memory

3.5 Multi-Evidence Fusion

3.6 Phys-Mem Scheduler

4 Experiments

4.1 Setup

4.2 Main Results

4.3 Ablation

4.4 Efficiency

4.5 Failure Analysis

5 Conclusion

Appendix

Supplementary

十、Git建议

commit:

Stage11

Paper Ready for Submission

对整个 12 个阶段的一个建议（也是我认为最值得调整的地方）

现在这 12 个 Stage 已经形成了一条完整的科研路线，但如果你的目标确实是 CCF-B/B+，我建议再增加一个贯穿所有阶段的固定要求：

每完成一个 Stage，不要立刻进入下一个 Stage，而是完成一次"阶段验收（Stage Review）"。

建议每个 Stage 都输出一份统一格式的 Review：

Stage Review

1. 本阶段解决了什么问题（Research Question）
2. 本阶段的核心假设（Hypothesis）
3. 修改了哪些模块（Implementation）
4. 是否保持了公平对比（Fair Comparison）
5. 可以做哪些消融实验（Ablation）
6. 目前还存在哪些理论缺口（Limitation）
7. 为什么需要进入下一阶段（Next Motivation）

这样最终你得到的不仅是一套开发 Prompt，而是一套完整的科研开发闭环：每个阶段都有研究动机 → 方法 → 实现 → 实验 → 局限 → 下一阶段动机，整个故事会非常自然，也更符合高质量论文的组织方式。

给出stage12

我建议Stage12 不要叫"结束阶段"，而应该叫：

Stage12: Research Extension（论文扩展与下一代框架）

因为对于CCF-B/B+论文来说，Reviewer最后最喜欢看的就是 Future Work。

如果你的论文到 Stage11 就结束了，那么 Reviewer 会觉得：

"只是完成了一个工程。"

但是如果有 Stage12，那么 Reviewer 会觉得：

这是一个可以持续发展的 Framework。

实际上，Stage12 是把整个 Phys-Mem 从一个算法提升成一个 Framework。

Stage12 Research Extension (Framework Evolution)

一、阶段目标 (Goal)

这一阶段：

不是：

继续优化论文。

不是：

继续调参数。

而是：

重新站在：

Framework Designer

的角度。

思考：

Phys-Mem还能继续发展什么？

目的：

让论文：

Future Work

变得：

可信。

二、本阶段论文目标

对应：

Future Work

Discussion

Open Problems

同时：

整理：

整个：

Framework。

让：

Phys-Mem：

真正成为：

Memory Framework。

而不是：

Depth Trick。

三、本阶段开发目标

停止：

所有：

算法开发。

开始：

Framework设计。

即：

设计：

下一代：

Memory Interface。

例如：

目前：

Similarity

↓

Fusion

↓

Scheduler

未来：

可能：

Similarity

↓

Evidence Pool

↓

Reasoning

↓

Scheduler

或者：

Perception

↓

Memory Graph

↓

World State

↓

Scheduler

四、允许修改范围

允许：

新增：

docs/

framework/

future/

design/

roadmap/

允许：

新增：

接口文档。

设计文档。

未来规划。

五、禁止修改

禁止：

继续：

训练。

修改：

实验。

改：

论文结果。

保证：

论文：

Version 1.0

冻结。

六、Codex Prompt（完整）

任务：

开始：

Framework Evolution。

当前：

整个Phys-Mem已经完成。

论文已经完成。

实验已经完成。

不要继续修改任何算法。

=====

目标：

重新设计：

Phys-Mem。

把它：

从一个方法。

升级成：

Memory Framework。

=====

请分析：

目前：

Phys-Mem：

有哪些限制？

例如：

是否依赖：

Depth?

是否依赖：

Action?

是否依赖：

Rule?

=====

请思考：

未来：

有哪些方向？

例如：

VGGT

DUSt3R

Metric3D

SAM2

Video Foundation Model

World Model

Memory Graph

Large Action Model

等等。

=====

设计：

下一代：

Memory Interface。

要求：

保持：

模块化。

可扩展。

统一接口。

=====

设计：

Evidence Registry。

支持：

未来：

任意Evidence。

例如：

Appearance

Semantic

Geometry

Intent

Language

Audio

Physics

都可以注册。

=====

设计：

Reasoning Layer。

要求：

不要：

简单Fusion。

支持：

Rule。

Graph。

LLM。

RL。

Reasoning。

=====

设计：

Memory Graph。

支持：

Temporal。

Spatial。

Semantic。

Physical。

=====

最后：

请生成：

Framework Roadmap。

包括：

Version1：

StableWorld。

Version2：

Phys-Mem。

Version3：

Memory Graph。

Version4：

World Memory。

=====

最后：

生成：

Future Work。

要求：

符合：

CCF-B/B+

论文标准。

七、最终输出

Codex：

最终输出：

- ① Framework Architecture
- ② Future Memory Interface
- ③ Evidence Registry
- ④ Memory Graph
- ⑤ Framework Roadmap
- ⑥ Future Work
- ⑦ Open Problems
- ⑧ 下一代系统设计

八、Checklist

必须能够回答：

- ☒ Phys-Mem有哪些局限？
- ☒ 为什么需要Evidence Registry？
- ☒ 为什么Fusion不是终点？
- ☒ 为什么未来需要Memory Graph？
- ☒ World Model如何接入？

如果回答不了：

Stage12不能结束。

九、论文对应

对应：

Discussion

Future Work

Supplementary

Reviewer：

最喜欢：

这里。

十、Git建议

commit:

Stage12

PhysMem Framework v1.0 Released

建议打一个正式标签：

git tag

v1.0-paper

这样论文对应的代码版本固定，后续研究可以从该版本继续演化。

十一、整个项目最终成果

完成 Stage12 后，你的项目不再只是一个代码仓库，而应形成一套完整的科研资产：

Phys-Mem/

- |—— code/ # 最终算法实现
- |—— configs/ # 实验配置
- |—— checkpoints/ # 模型权重（可选）
- |—— datasets/ # 数据集说明
- |—— evaluation/ # 实验平台
- |—— figures/ # 论文图片
- |—— tables/ # 实验表格
- |—— docs/
 - | |—— architecture.md # 系统架构
 - | |—— framework.md # Phys-Mem框架设计
 - | |—— roadmap.md # 技术演进路线
 - | |—— future_work.md # 未来工作
- |—— paper/
 - | |—— paper.tex
 - | |—— paper.pdf
 - | |—— supplementary.pdf
 - | |—— rebuttal_notes.md
- |—— scripts/ # 实验脚本
- |—— README.md
- |—— LICENSE

我对整个 13 个阶段（Stage0~Stage12）的一个建议

经过前面的讨论，我认为目前最大的提升空间已经不是 Prompt，而是研究路线本身。

我建议把整个项目最终凝练成一条非常清晰的科研主线：

Stage0-2

Understand

（理解）

↓

Stage3

Refactor

（解耦）



Stage4-5

Semantic

（二维语义）



Stage6

Geometry

（三维几何）



Stage7

Intent

（动作意图）



Stage8

Evidence

（多源证据）



Stage9

Decision

（Phys-Mem 调度）



Stage10

Evaluation

（实验验证）



Stage11

Paper

（论文撰写）

↓

Stage12

Framework

（未来扩展）

这条路线已经不只是一个开发计划，而是一套完整的科研方法论。每个阶段都对应一个明确的研究问题（Research Question）、一个技术模块（Engineering Module）和一节论文内容（Paper Section），最后自然形成从 Baseline → 创新 → 实验 → 论文 → Framework 的完整闭环。这种组织方式会比单纯围绕 ORB、LightGlue 或 Depth 展开，更容易形成一篇结构完整、逻辑严密的 CCF-B/B+ 论文。